ELP: Empirical Lane Parallelism

A method for software engineering with multiple agents in parallel, orchestrated by humans

Eduardo Díaz Cortés

2026-05-13

Contents

ELP: Empirical Lane Parallelism . . . 1
Abstract . . . 2
1. Motivation . . . 2
2. Definitions . . . 2
3. Project starting point . . . 3
4. Roles . . . 4
5. Anatomy of a lane . . . 6
6. The brief as contract . . . 6
7. Empirical capture . . . 7
8. Operational patterns . . . 9
9. Absolute rules . . . 10
10. Anti-patterns . . . 10
11. When to apply and when not . . . 11
12. Comparison with neighboring methods . . . 11
13. Metrics . . . 12
14. Case study: a compiler for a functional language with algebraic effects . . . 13
15. Known limitations . . . 15
16. Origin and references . . . 15
Appendix A. Minimal brief template . . . 16
Appendix B. Minimal lane retro template . . . 17

ELP: Empirical Lane Parallelism

A method for building software with multiple AI agents working in parallel, under human orchestration, with empirical capture of the process.

"Welcome back, my friends, to the show that never ends."

Emerson, Lake & Palmer, *Karn Evil 9: First Impression Part 2*, 1973.

Abstract

ELP (*Empirical Lane Parallelism*) is a software engineering method that combines three ideas:

1. **Parallel lanes.** Each unit of work runs as an isolated session: a *git worktree*, its own branch, its own `tmux` session. Several lanes run in parallel.
2. **Brief as contract.** Each lane receives a structured document defining its scope, the decisions already made, what to do and what not to do, and what to deliver.
3. **Empirical capture of the process.** Pre-code audits, mandatory per-lane retrospectives, and per-build measurements are kept as a dataset that feeds the next lanes.

ELP does not replace the classical development methods, it complements them with an explicit pattern for coordinating several agents without the system degrading within a few weeks.

The method has been empirically validated on *greenfield* projects, that is, new projects with no legacy code that start with a written product vision and general design but no prior implementation. ELP extends naturally to projects with a product vision and a well-defined *backlog*, although sustained practice outside the *greenfield* case remains future work.

---

1. Motivation

Working with AI assistants in a single conversation works well for small tasks. As the project grows, the pattern breaks:

- **Context fills up.** The agent loses relevant details that were there at the start.
- **Changes step on each other** when several tasks interleave.
- **No record remains** of what was tried and why it was discarded.
- **Every conversation starts from zero**, with no access to prior learning.

The methods of the current spectrum (*vibe coding*, *Pantser*, SDD, SPDD, BMAD) do not provide an operational answer to the problem when combined with scale (tens of PRs per day) and duration (multi-month projects). ELP aims to fill that gap.

---

2. Definitions

Lane. A parallelizable unit of work, tied to a single *backlog* item (typically a card on a Kanban board or a GitHub *issue*). Each lane is materialized as a *git worktree* with its branch, its brief, and its `tmux` session. The agent that executes a lane runs in **autonomous mode** (in Claude, this corresponds to *auto mode on*): it executes the brief from start to finish without pausing for confirmation, except at the *STOP-and-report* points explicitly declared.

Brief. A structured document that defines the lane's contract. It is the input of the agent that executes the lane. The name comes from *briefing*: before launch, the architect and the integrator hold a *briefing*, an explicit alignment, and the resulting document is what the agent receives as a contract. The brief **complements** the *backlog* item (which captures the functional *what*) with tactical instructions (the operational *how* for that specific execution).

Kanban. A board (physical or digital) where the architect manages initiatives, triages priorities, and decides what to address in the next lane. ELP does not prescribe the specific tool. In the initial practice, GitHub *issues* were used as a functional substitute. What matters is that there is a single,

consultable place where the *what to build* lives separate from the *how to build* (which lives in the brief).

Integrator. An AI agent with an interactive session that holds the shared context with the human, reads the repository state, writes the briefs, launches the lanes, and monitors their progress. It does not touch code directly.

Architect. The human. Decides the scope, the order of the lanes, integrates the PRs, authorizes irreversible actions, and corrects the integrator or the lanes when they go wrong.

Audit (*Phase 0*). A purely documentary lane that validates the premise of a milestone with data before any code is written. It can discard the entire proposal.

Lane retro. A document that each lane writes **before** opening the PR, with metrics, errors found, specification ambiguities, and friction points.

Build TSV. A tab-separated file where every *build* or *test* invocation is logged with timestamp, command, outcome, and elapsed time.

***STOP-and-report*.** The behavior of the lane agent when it encounters a situation outside its scope: it stops and reports, it does not improvise.

---

3. Project starting point

ELP does not start from zero. It assumes explicit initial conditions, without which the method does not hold. The empirically validated scope is **greenfield**. Its extension to projects with a pre-existing *backlog* is natural, but it requires the same preconditions.

3.1 Pre-existing documentation

The project starts with a minimum body of documentation that the integrator can read and that serves as the shared source for writing the briefs. Typically it includes:

- **Product vision.** What is to be built, for whom, and what problem it solves.
- **General design or target architecture.** Main components, language, runtime, technology decisions.
- **Examples and prototypes.** Snippets of the target syntax, interface mockups, a *spike* on a critical piece, reference repositories.
- **Glossary or domain model.** Problem-domain terms and how they relate to each other.

The quality of the briefs is a direct function of this documentation. Without a written vision, the integrator invents the scope. Without a general design, the lanes drift in incompatible directions.

3.2 Pinned principles

Before the first lane the following are established, ideally in writing in the repository (`docs/principles.md` or equivalent):

- **Architectural principles.** Invariants that no lane is allowed to break. For example, *"selfhost byte-identical on every PR"*, *"no runtime dependency on X"*, *"every observable function must be instrumented"*.
- **Business principles** (when they apply). What the product prioritizes when tradeoffs arise. For example, *"latency over throughput"*, *"legacy compatibility over release velocity"*.

- **Time constraints.** Launch goals and dates, release roadmap, freeze windows, externally committed milestones.

The principles act as axioms of the flow. The brief cites them, it does not re-discuss them. Changing a principle is a conscious decision of the architect, not something a lane can do in passing.

3.3 Backlog managed on Kanban

The *what to build* lives on a **Kanban board** that the architect uses to manage initiatives and triage what to address next. ELP does not prescribe the specific tool. It can be a dedicated board (Linear, Jira, Trello, GitHub Projects) or the set of GitHub *issues* as a functional substitute (ELP's initial implementation used the latter).

What matters is that the board fulfills three functions:

1. **Inventory.** Each item is a candidate unit to become a lane.
2. **Prioritization.** The architect orders, groups, and discards items. The integrator reads that order.
3. **Traceability.** Every brief, retro, and PR references the Kanban item it comes from.

The brief **complements** the Kanban item. The item captures the functional *what*, with the detail the architect deems necessary, and the brief adds the tactical *how* (pinned decisions, files to touch, explicit scope) as the result of the *briefing* between architect, integrator, and lane agent.

3.4 Greenfield versus projects with a pre-existing backlog

Dimension	Greenfield (validated)	With pre-existing backlog (natural extension)
Starting code	none or an initial *spike*	existing base with conventions
Product vision	freshly written	mature, possibly versioned
Architectural principles	established at the start	documented by extracting them from the code
Kanban board	new, filled over time	pre-existing, with given debt and priorities
Main risk	drifting without guides	imposing ELP on a flow that already works

The extension to non-*greenfield* projects requires a prior step: **making the implicit explicit** (principles, conventions, unwritten contracts) so that the briefs can cite them.

---

4. Roles

ELP defines three roles with clear, non-overlapping responsibilities.

4.1 Architect (human)

Responsibilities:

- Decide the scope of each milestone and the order of execution across lanes.
- Approve or reject PRs, or enable automatic integration under specific conditions.

- Explicitly authorize irreversible actions (*force push*, destructive cleanup, *push* to *main*).
- Correct when the integrator or the lanes go wrong.
- Make the final call when there is disagreement.

What the architect typically **does not** do:

- Write code line by line.
- Re-discuss decisions already pinned in the briefs.
- Approve PRs without the lane retro being written.

4.2 Integrator (AI agent, interactive session)

Responsibilities:

- Hold the shared project context, reading the repository state (*issues*, PRs, prior retros, audits).
- Write detailed briefs before launching each lane.
- Launch the lane agents in separate `tmux` sessions, each with its *worktree*.
- Monitor progress using `tmux capture-pane` and *wakeup* scheduling.
- Surface critical findings to the human.

What the integrator **does not** do:

- Touch the lane code directly.
- Make irreversible decisions without human authorization.
- Clean up *worktrees* or sessions without explicit authorization.

4.3 Lane (AI agent, isolated session, autonomous mode)

The lane agent runs in **autonomous mode from start to finish**. In Claude's case, this corresponds to operating with *auto mode on*. The agent does not request interactive confirmation between brief steps, executes the whole chain (explore, diagnose, implement, validate, write the retro, *commit*, *push*, and open the PR), and only stops at the *STOP-and-report* points explicitly declared in the brief, or when an irreversible action requires authorization (see §9).

Autonomy is what makes parallel execution viable. A human cannot supervise N lanes at once. The brief replaces continuous supervision. That is why its quality is the ceiling on the lane's quality.

Responsibilities:

- Receive a brief at startup and read it in full before acting.
- Work in its isolated *worktree*, on its own branch.
- Execute the flow *explore, diagnose, implement, validate, write the retro, commit, push, and open the PR* without pausing for confirmation.
- Enable automatic integration when green CI is expected.
- Perform ***STOP-and-report*** on design ambiguities that exceed the brief.

What the lane **does not** do:

- Improvise scope decisions.
- Touch files outside the scope declared in the brief.
- Clean up its own *worktree* or session.

---

5. Anatomy of a lane

5.1 File layout

Each lane lives in three places:

```
/tmp/wt-claude-prompt-<issue>-<descr>.txt   # the brief
/tmp/launch-<issue>.sh                      # script that invokes the agent
~/work/<project>.<issue>-<descr>/           # the worktree with its branch
```

A `tmux` session named `<issue>-<descr>` runs the agent.

5.2 Launch

```
chmod +x /tmp/launch-NNN.sh
tmux new-session -d -s issue-NNN-descr \
    "wt switch --create issue-NNN-descr -x /tmp/launch-NNN.sh"
```

`wt switch --create` creates the *worktree* together with the branch, enters the directory, and runs the script. The script invokes `exec claude "$(cat brief)"` and starts the agent with the brief as the initial input.

5.3 Monitoring

```
tmux list-sessions
tmux capture-pane -t <session-name> -p | tail -30
```

If the trajectory needs correction, use `tmux send-keys` to preserve the context the agent already has. **Re-briefing from scratch loses the learning accumulated in the session.**

5.4 Cleanup after integration

Only after the lane's PR has been integrated:

```
tmux kill-session -t issue-NNN-descr
git worktree remove ~/work/<project>.issue-NNN-descr
git branch -D issue-NNN-descr
git pull --ff-only origin main
```

Pinned rule: cleanup requires explicit authorization from the architect. Premature cleanup has destroyed useful context in practice.

---

6. The brief as contract

The brief is the difference between an agent that delivers a review-ready PR and one that wastes time. It is the product of the *briefing* between architect, integrator, and, in its final form when the lane is launched, the agent that executes it. It is an explicit alignment on what is going to be done, with which decisions already closed, and under which constraints. The canonical structure is as follows:

6.1 Goal

One to three sentences. What is delivered and why it matters now. Ideally it cites the *issue* number.

6.2 Context

- Repository and branch (always `issue-NNN-descr` from the current `main`).
- Expected verification (for example, `git log --oneline -5 origin/main`).
- Project pinned rules (`CLAUDE.md`, files that are not to be touched).

- Applicable architectural and business principles (see §3.2).
- Parallel lanes in flight, to anticipate conflicts.

6.3 Read first

List of files the agent must read before writing code:

- The *backlog* item (*issue* or Kanban card) in full.
- The relevant audit or design document, when it exists.
- Retros of prior lanes that share context.
- Specific source files the lane will touch.

6.4 Pinned decisions

Decisions already made that the agent **must not re-discuss**. This section is critical. The AI tends to "improve" what was already decided. Typical examples:

- *"Do not fix issue #219 in this lane."*
- *"Do not touch `CHANGELOG.md` or `VERSION`. The release bump regenerates them."*
- *"Smallest-delta wins. Change the minimum needed to make it pass."*
- *"Diagnose before implementing. The first step is to understand the failure."*

6.5 Lane scope (in and out)

An explicit list of what to DO and what NOT to do.

What NOT to do is as important as what TO do. The model is trained to be helpful. If it is not told to stop, it keeps going. Typical cases:

- Do not touch other modules.
- Do not touch versioning files (`CHANGELOG`, `VERSION`).
- Do not open new *issues* for adjacent gaps (keep the scope separate).
- *STOP-and-report* on conditions X.

6.6 Required outputs

- Expected modified files.
- *Commit* message in the project format (typically *Conventional Commits*).
- PR body with what to include.
- Closing commands: `gh pr create`, `gh pr merge --auto`.

6.7 Discipline reminders

- Required test tier before *push*.
- Clean working tree at every *commit*.
- Automatic integration enabled when applicable.

6.8 Instrumentation (mandatory)

- Build logging to TSV.
- Retro document at `docs/lane-experience-<lane>.md` with fixed sections (see §7.2).

---

7. Empirical capture

The formal methods of the current spectrum (SDD, SPDD, BMAD) produce artifacts. ELP **also measures the process**. Without this, the flow degrades within a few weeks.

7.1 Phase 0 audits

When a milestone spans several zones or assumes an unvalidated hypothesis, a **purely documentary** lane is run that validates the premise with data before any code is written.

Audit structure (`docs/<milestone>-phase0-audit.md`):

1. Empirical inventory (searches, counts, and scans over the current code).
2. Surface quantification (references per file, types of change).
3. Survey of options (A, B, C, D) with cost, risk, and resulting state for each.
4. Quality-gate strategy (how the build stays green at every PR).
5. Risk inventory.
6. Verdict and recommended phase order.

Typical result documented in practice: some audits discard the entire proposal with empirical data, saving weeks of work on a wrong milestone.

7.2 Lane retros

Each lane writes `docs/lane-experience-<lane>.md` **before** the PR. **No retro, no PR.** Fixed structure:

- **Objective metrics.** Start and end timestamps, counts of *build* and *test* invocations.
- **Change inventory.** Files modified, lines added and deleted, sites migrated.
- **Compiler errors encountered.** By error class, where they appeared, the fix applied, and the failed attempts.
- **Friction points.** Answers to the specific questions the brief asked.
- **Ambiguities or interpretive choices.** Where the agent had to decide without the brief telling it.
- **Subjective summary.** Confidence, the hardest part, the easiest part, what helped or hindered the tooling.
- **Limitations of this report.**

The retros serve four purposes:

1. They capture information while the context is fresh.
2. They are input for the next lane (lessons learned).
3. They are a dataset for honestly evaluating *LLM authorability*, that is, which portion of the work the agent can actually complete.
4. They document gaps that turn into new *issues*.

7.3 Measurements (TSV)

Before the first invocation of `make` or equivalent, the brief instructs:

```
export LANE="issue-NNN-descr"
date -Iseconds > /tmp/lane-${LANE}-start.txt
echo -e "timestamp\tcmd\toutcome\telapsed_s" \
    > /tmp/lane-${LANE}-builds.tsv
```

Each `make` is logged as a TSV line. When the lane closes, this content is appended to the retro.

The TSV captures:

- The lane's real wall-clock time.
- How many retries each test tier needed.
- Time spent on debugging versus pure implementation.

It is the dataset that allows honestly evaluating how much each lane costs and comparing similar lanes.

7.4 Honesty targets

Pinned documents (`docs/<feature>-honesty-targets.md`) that record where the system's behavior is **simulated** and where it is **real**. For example:

> *"0 fires in self-compile, here is why: the typer pre-increfs before the check."*

This does not inflate the metric. It reports what was measured, not what was projected. The culture is verifiable against the TSV and the retros.

7.5 Findings as new backlog items

When an agent identifies a gap outside scope, the flow is:

1. Document it in the *friction points* section of the retro.
2. **Do NOT fix it.** *STOP-and-report*.
3. The integrator decides whether to create a new Kanban item (*issue* or card) now.
4. If created, the new item's body cites the retro as source.

Each new gap is **its own lane**. This prevents a lane from turning into an infinite chain of incidental work.

---

8. Operational patterns

8.1 Parallel and sequential lanes

Parallel when:

- They touch different zones of the system.
- They do not share consumer files.
- Their outputs are orthogonal.

Sequential when:

- They touch the same consumer files.
- One establishes a pattern that the next replicates.
- The next one needs the **retro** of the previous one.

Operational criterion: do they collide in the work area, or in the learned lesson?

8.2 STOP-and-report

The brief always includes explicitly an instruction of the form *"if you find X, stop and report"*. Typical cases:

- A bug adjacent to scope.
- A design decision the brief does not cover.
- The need to touch more zones than anticipated.

The agent **does not improvise scope decisions**. It reports and waits.

8.3 Wakeup scheduling

To avoid requiring constant human polling and not to leave orphaned lanes, the integrator is scheduled to wake up at defined intervals.

Operational rules:

- The typical delay ranges from 60 to 3600 seconds.
- Skip the 60 to 270 second range if there is no urgency. The prompt cache lasts 5 minutes. Shorter waits lose the cache.
- The *wakeup* prompt must be self-contained. When it fires, it has to be interpretable without prior context.

8.4 Automatic integration on green CI

When branch protection requires specific green checks, the lane enables automatic integration on every PR. When the checks pass, the integration fires by itself.

Learned exceptions:

- Purely documentary PRs can get blocked if `paths-ignore` skips the workflow and the check is never generated. The solution is a self-skipping workflow that reports success without running anything when it should be skipped.
- Non-required sub-checks may be red and the PR still integrates. The lesson is to run all quality gates **locally** before *push*. Do not rely on CI alone.

---

9. Absolute rules

Non-negotiable rules that apply to every actor in the flow:

1. **Do not touch versioning files** (`CHANGELOG`, `VERSION`). The release bump regenerates them.
2. **No direct push to `main`**. Always PR with automatic integration.
3. **Without explicit authorization, no `worktree remove`, no `branch -D`.**
4. **Do not compromise the user's work.** Investigate before deleting.
5. **Mandatory retros** for every lane.
6. **Phase 0 audits** for large milestones (more than one PR, unvalidated hypothesis, several zones affected).
7. **Honesty over optimism.** Report what was measured, not what was projected.
8. **Detailed briefs.** A thin brief produces a lost agent.
9. **Pinned principles are not re-discussed inside a lane** (§3.2). Changing a principle is a conscious decision of the architect.

---

10. Anti-patterns

Patterns documented as problematic that ELP explicitly avoids:

- **Premature documentation files** (TODOs, plan.md, decision.md). The *issues* and the retros cover what should persist. The rest lives in the conversation.
- **Re-fixing a bug outside scope.** Open a new *issue*, do not extend the lane.
- **Migration with undocumented surviving aliases.** Each live alias needs a pinned cause in the PR body.

- **Automatic integration without locally verifying the checks.** Non-required sub-checks can lie.
- **Thin briefs.** *"Implement X"* without context produces a lost agent.

---

11. When to apply and when not

11.1 ELP fits when

- The project will last months, not days.
- The **preconditions of §3** are in place: a written vision and general design, pinned architectural principles, clear time constraints, and a Kanban (or equivalent) being actively managed.
- There is scope for several PRs in parallel, over orthogonal zones of the system.
- There is value in preserving learning across lanes.
- The architect can dedicate time to writing briefs.

The method was validated on *greenfield* projects. Its extension to projects with a pre-existing *backlog* requires first making the implicit explicit (principles, conventions, unwritten contracts) so that the briefs can cite them (§3.4).

11.2 ELP does NOT fit when

- It is a throwaway prototype. The overhead of briefs and retros does not amortize.
- It is an urgent production fix. The cadence of parallel PRs does not apply.
- The team has no mature tests or CI. ELP **amplifies** what already exists. Without a solid pipeline, it amplifies the chaos.
- The architect cannot dedicate 10 to 20 minutes per brief. The quality of the brief is the ceiling on the quality of the lane.
- There is no product vision or general design beforehand. ELP does not invent the *what to build*, it orchestrates it.

---

12. Comparison with neighboring methods

12.1 ELP versus BMAD-METHOD

BMAD divides by **functional specialty**, with 12+ specialist agents (product manager, architect, developer, UX, QA, etc.) that the human activates across agile phases. ELP divides by **parallelizable unit**, with generalist lanes and bounded scope, each in its own session.

Dimension	BMAD	ELP
Division of work	by specialty	by parallelizable unit
Topology	sequential by phases, with multi-agent collaboration inside the session (*party mode*)	several parallel and independent lanes, in isolated sessions
Central artifact	PRD, architecture document, user stories	brief and retro per lane
Source of *what to build*	PRD produced by the product-manager agent	items of the *backlog* managed on Kanban
Process capture	partial	persistent (audits, retros, and TSV)
Change isolation	one branch, inside the IDE	one *worktree* and one `tmux` session per lane

12.2 ELP versus SPDD

SPDD treats the prompt as a **living artifact**, versioned in sync with the code, with templates like the *REASONS Canvas* to force clarity. ELP treats the **brief** as a single-use contract, but accumulates learning in the **retros**, which are versioned. They are orthogonal and can be combined: nothing prevents using the *REASONS Canvas* as the structure of an ELP lane brief.

12.3 ELP versus SDD

SDD treats the **specification** as a persistent artifact and source of truth: the code is considered a derivative that can be regenerated from the spec, and requirement changes become systematic regenerations. ELP inverts the priority: the integrated code is the persistent artifact, the brief is a single-use tactical contract, and the **retro** accumulates the learning that the disposable spec in SDD does not capture. The two views are complementary: a living spec in the SDD style can coexist with ELP lanes that implement it in parallel.

12.4 ELP versus Pantser

The term *Pantser* applied to software development was articulated by Eduardo Díaz in *Mapas o Brújulas* (lnds.net, 2025), borrowed from literature. The *Pantser* is the one who writes **by the seat of his pants**, without a rigid map, adjusting direction with every step. The classical counterpart is the *Plotter*, who plans meticulously before executing.

The *Pantser* iterates empirically with disciplined human review, but the learning **dies with the conversation**. ELP captures that learning in persistent formats that survive session close. The central difference is the **persistence of learning**, not the presence of empiricism.

---

13. Metrics

Metrics ELP recommends capturing at the project level:

- **PRs integrated** per day and per week.
- **Parallel lanes at peak** of simultaneous concurrency.

- **Percentage of lanes ending in useful *STOP-and-report***, as an indicator of brief quality.
- **Wall-clock time per lane** versus estimate, from the TSV.
- **Retries per lane**, separating real debugging from first-time implementation.
- **Phase 0 audits run** and **percentage that discarded the milestone**.
- ***Issues* opened as follow-up** from the retros, as a flow-health signal.

---

14. Case study: a compiler for a functional language with algebraic effects

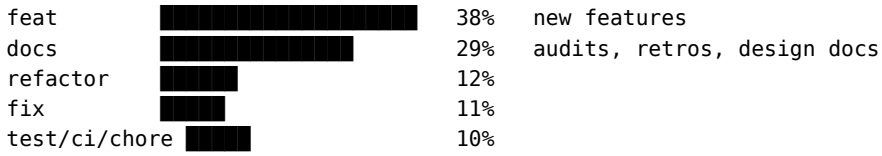
ELP was formalized from accumulated practice on a concrete project: building from scratch a **self-hosted functional programming language with algebraic effects** and an LLVM *backend*, executed between **April 20 and May 4, 2026**, over 14 consecutive days. This project is *greenfield* and started with a product vision, a general design, examples of target syntax, and architectural principles pinned from day zero (§3).

14.1 Raw figures

Indicator	Value
Total time	**14 consecutive days**
Commits	**868**
Pull requests integrated	**144**
Issues closed	**69**
Releases	v0.1.0 to **v0.31.0**
Peak *commits* in a day	**135**
Phase 0 audits run	3
Lane retros produced	more than 25
Peak of parallel lanes	3 simultaneous

The figures are not perception. They come from `git log`, from `gh pr list`, and from the TSV of each lane. Every PR cites the retro that precedes it.

14.2 Commit distribution



29 % docs is high for a compiler. It is exactly the empirical dataset that ELP recommends: phase 0 audits, retros, design documents per feature, honesty targets. **11 % fix is low**. The features arrived clean thanks to the quality gates (selfhost byte-identical from day 1, mandatory tier 1 tests before *push*, mandatory retro before the PR).

> Paying the 29 % docs upfront is what keeps the 11 % fix.

14.3 What was built

A self-hosted functional language with properties rarely found together in a single implementation:

- **Three-stage compiler.** Stage 0 in C (around 10K lines), stage 1 as a bridge (around 6K), stage 2 self-hosted (around 41K lines).
- **Selfhost byte-identical** as a quality gate from day 1. Every change must be able to recompile itself byte-for-byte.
- **Effect system** Effekt-style: capabilities, handlers, row inference, effects like `Console`, `Spawn`, `Cancel`, `Actor`, `Mutable`, `Ffi`.
- **Perceus-style reference counting** (Koka). Reference counting compiled at compile time, with no borrow checker, with reuse-in-place and elimination of redundant increfs and decrefs.
- **BEAM-style fibers.** Per-fiber private heap, copied messages, cooperative scheduler with `swapcontext`.
- **Complete standard library and tooling:** `kai build/run/test/fmt/repl/lsp/doc/bench/check`.
- **Mandatory tail-call optimization.** It is a language rule. Self-tail-calls are rewritten as `goto` loops in the C emission.

14.4 Industry comparison

Comparable public projects and their time to functional self-hosting:

Project	Time	Team
Roc (effects and functional)	around 5 years	research
Koka (invented Perceus)	over 10 years	academic
Effekt (capabilities)	around 5 years	PhD
Crystal (LLVM and inference)	around 3 years	core team

Human equivalent to reach the same technical scope:

> **From 3 to 4 senior compiler-expert engineers, over 12 to 18 months, equivalent to between USD 1.0 and 1.5 million in payroll.**

ELP reached the same point in **12 to 14 days, with one human (the architect), one integrator agent, and parallel lanes**.

14.5 What of the method contributed

Honest attribution of the result, without extrapolating:

1. **Selfhost byte-identical from day 1.** Any change that breaks self-compilation is discovered in the build, not in production. It shielded every feature against invisible regressions.
2. **Three-stage bootstrap.** Allows stage 2 to use the full language without being limited to the subset that a C-only compiler could implement. Stage 1 acts as a bridge.
3. **Phase 0 audits as culture.** Three audits already saved weeks. One fully discarded a bootstrap-split proposal with empirical data. The premise was false. Without the audit, the team would have implemented on top of a wrong hypothesis.

4. **Retros as a dataset, not as documentation.** Retros that record compiler errors, failed fix paths, and friction points feed the next lane and are the actual data for evaluating *LLM authorability*.
5. **Pinned honesty targets.** Each feature documents where its behavior is simulated and where it is real. For example, *"0 fires in self-compile, here is why."* The culture scales because honesty is verifiable against the TSVs.
6. ***Backlog* as roadmap.** The functional *what to build* lives in the Kanban (in this case, GitHub *issues* as a functional substitute), separate from the *how*, which lives in the brief. This prevents each lane from becoming a chain of incidental work.

14.6 What was closed as a negative

Decisions that did not land, pinned as *negatives*:

- *Drop specialisation*. Measured a –1.7 % at `-O2` and a +5.4 % regression at `-O0`. The lane was closed as a negative. Phase 2 unboxing had already absorbed the overhead.
- *Bootstrap split*. Discarded by an empirical audit before being implemented. Phase 0 saved 2 to 3 weeks of wrong work.

These negatives are not failures, they are **data**. Each has a pinned retro that prevents repeating it.

14.7 Final observation

The project has a rarely-found property. It is **technically ambitious and operationally honest** at the same time. Phase 0 audits, retros, and honesty targets build a dataset that a human team usually postpones or loses. The human, the architect, did not write code. They kept the judgment over scope, the architecture decisions, and the calls of caution of the kind "this does not convince me". The agents executed the lanes with detailed briefs.

If the underlying hypothesis holds (*LLM authorability* validable against the accumulated dataset), this case is the first empirical evidence documented of a technically complex project built under ELP.

---

15. Known limitations

- If the architect is sleeping and an agent issues a *STOP-and-report*, the lane stays **blocked** until the human responds. A practical mitigation is to include in the brief pre-approved authorizations for common cases.
- If CI takes long and the *wakeup* fires earlier, prompt cache is spent without progress. It is a conscious tradeoff.
- Detailed briefs take between 10 and 20 minutes from the architect. It is not always worth it for trivial lanes, like a single-file edit.
- The method assumes mature versioning and CI tools. In projects without them it does not apply directly.
- Changing machines or restarting the integrator's session loses the live context. Today there is no structural fix.

---

16. Origin and references

ELP was formalized from accumulated practice in long-duration projects with multiple AI agents working in parallel. The first public articulation of the method appears in this document.

Related methods and references:

- **Díaz, E.,** *"Mapas o Brújulas"*, lnds.net, 2025-10-31. URL: https://lnds.net/blog/lnds/2025/10/31/mapas-o-brujulas/. Spanish-language articulation of the *Plotter* and *Pantser* pair applied to software development.
- **BMAD-METHOD** (*Breakthrough Method for Agile AI-Driven Development*), by Brian Madison. Multi-agent pipeline with functional roles. Repository: https://github.com/bmad-code-org/BMAD-METHOD. Documentation: https://docs.bmad-method.org/. Official YouTube masterclass: https://www.youtube.com/watch?v=LorEJPrALcg.
- **SDD** (*Spec-Driven Development*). The specification as source of truth and persistent artifact from which the code is regenerated. Reference implementation: GitHub Spec Kit, https://github.com/github/spec-kit. Documentation: https://github.github.com/spec-kit/. Full tutorial on YouTube: https://www.youtube.com/watch?v=xfyec__ieHA.
- **SPDD** (*Structured Prompt-Driven Development*), by Wei Zhang and Jessie Jie Xia (Thoughtworks, 2026). The prompt as a living artifact, with the *REASONS Canvas* as a template. Article on martinfowler.com: https://martinfowler.com/articles/structured-prompt-driven/. Explanatory video on YouTube: https://www.youtube.com/watch?v=CotFOAGtb64.
- **Saltzer and Schroeder**, *The Protection of Information in Computer Systems*, 1975. Secure-design principles that apply equally to the agent.
- **Karpathy, A.,** *"Vibe Coding"*, 2024.
- **Krouse, S.,** *"Vibe Code is Legacy Code"*, 2025.
- **Emerson, Lake & Palmer**, *Karn Evil 9: First Impression Part 2*, 1973. Origin of the retroactive acronym ELP.

Appendix A. Minimal brief template

```
# Lane brief: issue-NNN-<descr>

## Goal
[1 to 3 sentences. What is delivered and why.]

## Context
- Repository: <repo>, branch: issue-NNN-<descr>
- Verification: git log --oneline -5 origin/main
- Project pinned rules: CLAUDE.md
- Parallel lanes in flight: [list]

## Read first
- Issue: gh issue view NNN
- Audit: docs/<milestone>-phase0-audit.md (if applicable)
- Related retros: docs/lane-experience-<previous>.md
- Files to inspect: <paths>

## Pinned decisions
- [decision 1]
- [decision 2]

## Lane scope
DO:
- [action 1]
- [action 2]
```

```
DO NOT:
- [prohibition 1]
- [prohibition 2]

STOP-and-report if:
- [condition 1]

## Required outputs
- Expected modified files: <paths>
- Commit message: <type>(<scope>): <subject>
- PR body: [what to include]
- Closing: gh pr create + gh pr merge --auto

## Discipline
- Tier 0 and tier 1 tests locally before push.
- Clean working tree at every commit.

## Instrumentation
- Build TSV at /tmp/lane-${LANE}-builds.tsv.
- Retro at docs/lane-experience-issue-NNN-<descr>.md (mandatory sections).
```

Appendix B. Minimal lane retro template

```
# Lane retro: issue-NNN-<descr>

## Objective metrics
- Start: <ISO timestamp>
- End: <ISO timestamp>
- Wall-clock: <duration>
- Build invocations: <count per tier>

## Change inventory
- Modified files: <list>
- Lines added and deleted: +X / -Y
- Sites touched: <count>

## Compiler errors encountered
- [Error class]: where, fix applied, attempts.

## Friction points
- [Brief question]: answer based on what was observed.

## Ambiguities or interpretive choices
- [Decision without explicit specification]: justification.

## Subjective summary
- Confidence: high, medium, or low.
- Hardest: ...
- Easiest: ...
- How the compiler or tooling helped or hindered: ...

## Limitations of this report
- [What this report does not cover.]
```